

Лабораторная работа №4

Цель работы: Получение практических навыков при использовании Win32 API для исследования процессов.

Задание для выполнения лабораторной работы

Составить следующую программу, которая:

- принимая дескриптор, имя или полное имя модуля, возвращает другие два элемента в своих выходных параметрах (выполнить задание для своей программы и для любой известной библиотеки),
- будет выполнять последовательно по шагам следующее:
 1. используя функцию GetCurrentProcessId определит идентификатор текущего процесса.
 2. используя функцию GetCurrentProcess определит псевдодескриптор текущего процесса.
 3. используя функцию DuplicateHandle и значение псевдодескриптора определит дескриптор текущего процесса.
 4. используя функцию OpenProcess определит копию дескриптора текущего процесса
 5. закроет дескриптор, полученный функцией DuplicateHandle
 6. закроет дескриптор, полученный функцией OpenProcess
- выдает список перечисления всех процессов, потоков, модулей и их свойства в системе.

Ход выполнения лабораторной работы

Программа выведет полную информацию о занятой и свободной физической и виртуальной памяти, составит карту процессов, создаст виртуальную память для каждого процесса.

1. Создать форму с элементами для вывода информации о занятой памяти;
2. Сделать снимок памяти функцией CreateToolhelp32Snapshot, первый параметр будет TH32CS_SNAPPROCESS, который указывает, что нам нужен снимок именно процессов;
3. С помощью функции Process32First и Process32Next занести информацию о процессе в структуру, а из структуры, соответственно, в таблицу;
4. Получить дескриптор процесса с помощью функции OpenProcess по его ID;
5. Используя функцию GetProcessMemoryInfo, получить информацию об используемой памяти выбранного процесса, которую также следует занести в таблицу.

Вывод информации о потоках выбранного процесса

1. Сделать снимок памяти функцией CreateToolhelp32Snapshot, первый параметр будет TH32CS_SNAPTHREAD, который указывает, что нам нужен снимок потоков;
2. С помощью функций Thread32First и Thread32Next занести информацию о процессе в структуру, а из структуры, соответственно, в таблицу;

3. В конце работы закрыть все открытые дескрипторы;
4. С помощью функции GetCurrentProcessId определить ID текущего процесса;
5. С помощью функции GetCurrentProcess определить псевдодескриптор;
6. С помощью функции DuplicateHandle определить дескриптор текущего процесса;
7. С помощью функции OpenProcess определить дескриптор текущего процесса.

Вывод информации о модулях текущего процесса

1. С помощью функции GetModuleFileName можно узнать модель текущего процесса;
2. С помощью имени модуля определяем его дескриптор с функцией GetModuleHandle.

Вывод информации о модулях всех процессов

1. Сделать снимок памяти функцией CreateToolhelp32Snapshot, первый параметр будет TH32CS_SNAPMODULE, который указывает, что снимок будет именно процессов;
2. Функции module32First и module32Next позволяют занести всю информацию о процессе в структуру, а из структуры, соответственно, в таблицу;
3. Закрыть все открытые дескрипторы.

Программный код

```
#include <windows.h>
#include <tlhelp32.h>
#include <tchar.h>
#include <sstream>

// Определения идентификаторов элементов управления
#define ID_EDIT_PID 101
#define ID_BUTTON_PROCESS_LIST 102
#define ID_BUTTON_MEMORY_INFO 103
#define ID_BUTTON_THREAD_LIST 104
#define ID_BUTTON_MODULE_LIST 105
#define ID_BUTTON_FIND_PROCESS 106

// Глобальные переменные
const TCHAR szWindowClass[] = _T("win32app");
const TCHAR szTitle[] = _T("Win32 Application");
HINSTANCE hInst;
HWND hEditOutput;
HWND hEditPID;

// Прототипы функций
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
void PrintProcessList(HWND hEdit);
void PrintMemoryInfo(HWND hEdit);
void PrintThreadList(DWORD processID, HWND hEdit);
void PrintModuleList(DWORD processID, HWND hEdit);
void FindProcess(DWORD processID, HWND hEdit);

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine,
int nCmdShow) {
    UNREFERENCED_PARAMETER(hPrevInstance);
    UNREFERENCED_PARAMETER(lpCmdLine);

    WNDCLASSEX wcex;

    wcex.cbSize = sizeof(WNDCLASSEX);
```

```

wcex.style = CS_HREDRAW | CS_VREDRAW;
wcex.lpfnWndProc = WndProc;
wcex.cbClsExtra = 0;
wcex.cbWndExtra = 0;
wcex.hInstance = hInstance;
wcex.hIcon = LoadIcon(hInstance, IDI_APPLICATION);
wcex.hCursor = LoadCursor(nullptr, IDC_ARROW);
wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
wcex.lpszMenuName = nullptr;
wcex.lpszClassName = szWindowClass;
wcex.hIconSm = LoadIcon(wcex.hInstance, IDI_APPLICATION);

if (!RegisterClassEx(&wcex)) {
    MessageBox(nullptr, _T("Call to RegisterClassEx failed!"), szTitle, MB_OK);
    return 1;
}

hInst = hInstance;

HWND hWnd = CreateWindow(szWindowClass, szTitle, WS_OVERLAPPEDWINDOW,
    CW_USEDEFAULT, CW_USEDEFAULT, 500, 600, nullptr, nullptr, hInstance,
    nullptr);

if (!hWnd) {
    MessageBox(nullptr, _T("Call to CreateWindow failed!"), szTitle, MB_OK);
    return 1;
}

ShowWindow(hWnd, nCmdShow);
UpdateWindow(hWnd);

MSG msg;

while (GetMessage(&msg, nullptr, 0, 0)) {
    TranslateMessage(&msg);
    DispatchMessage(&msg);
}

return (int)msg.wParam;
}

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam) {
    switch (message) {
    case WM_CREATE:
        hEditOutput = CreateWindow(_T("EDIT"), _T(""),
            WS_CHILD | WS_VISIBLE | WS_VSCROLL | ES_LEFT | ES_MULTILINE |
            ES_AUTOVSCROLL,
            10, 100, 480, 400, hWnd, nullptr, hInst, nullptr);
        hEditPID = CreateWindow(_T("EDIT"), _T(""),
            WS_CHILD | WS_VISIBLE | WS_BORDER | ES_NUMBER,
            10, 50, 100, 20, hWnd, (HMENU)ID_EDIT_PID, hInst, nullptr);
        CreateWindow(_T("BUTTON"), _T("Get Process List"),
            WS_CHILD | WS_VISIBLE | BS_PUSHBUTTON,
            120, 50, 150, 20, hWnd, (HMENU)ID_BUTTON_PROCESS_LIST, hInst, nullptr);
        CreateWindow(_T("BUTTON"), _T("Get Memory Info"),
            WS_CHILD | WS_VISIBLE | BS_PUSHBUTTON,
            280, 50, 150, 20, hWnd, (HMENU)ID_BUTTON_MEMORY_INFO, hInst, nullptr);
        CreateWindow(_T("BUTTON"), _T("Find Process"),
            WS_CHILD | WS_VISIBLE | BS_PUSHBUTTON,
            120, 80, 150, 20, hWnd, (HMENU)ID_BUTTON_FIND_PROCESS, hInst, nullptr);
        break;
    case WM_COMMAND:
        TCHAR strPID[32];
        DWORD processID;
        switch (LOWORD(wParam)) {

```

```

        case ID_BUTTON_PROCESS_LIST:
            PrintProcessList(hEditOutput);
            break;
        case ID_BUTTON_MEMORY_INFO:
            PrintMemoryInfo(hEditOutput);
            break;
        case ID_BUTTON_THREAD_LIST:
            GetWindowText(hEditPID, strPID, _countof(strPID));
            processID = _ttoi(strPID);
            if (processID > 0) {
                PrintThreadList(processID, hEditOutput);
            }
            else {
                MessageBox(hWnd, _T("Please enter a valid process ID."), szTitle,
MB_OK);
            }
            break;
        case ID_BUTTON_MODULE_LIST:
            GetWindowText(hEditPID, strPID, _countof(strPID));
            processID = _ttoi(strPID);
            if (processID > 0) {
                PrintModuleList(processID, hEditOutput);
            }
            else {
                MessageBox(hWnd, _T("Please enter a valid process ID."), szTitle,
MB_OK);
            }
            break;
        case ID_BUTTON_FIND_PROCESS:
            GetWindowText(hEditPID, strPID, _countof(strPID));
            processID = _ttoi(strPID);
            if (processID > 0) {
                FindProcess(processID, hEditOutput);
            }
            else {
                MessageBox(hWnd, _T("Please enter a valid process ID."), szTitle,
MB_OK);
            }
            break;
    }
    break;
case WM_DESTROY:
    PostQuitMessage(0);
    break;
default:
    return DefWindowProc(hWnd, message, wParam, lParam);
}
return 0;
}

// Реализация функции PrintMemoryInfo
void PrintMemoryInfo(HWND hEdit) {
    MEMORYSTATUSEX memInfo;
    memInfo.dwLength = sizeof(memInfo);
    GlobalMemoryStatusEx(&memInfo);

    std::wstringstream ss;
    ss << _T("Total physical memory: ") << memInfo.ullTotalPhys / 1024 / 1024 <<
_T(" MB\r\n");
    ss << _T("Free physical memory: ") << memInfo.ullAvailPhys / 1024 / 1024 << _T("
MB\r\n");
    ss << _T("Total virtual memory: ") << memInfo.ullTotalVirtual / 1024 / 1024 <<
_T(" MB\r\n");
    ss << _T("Free virtual memory: ") << memInfo.ullAvailVirtual / 1024 / 1024 <<
_T(" MB\r\n");
}

```

```

        SetWindowText(hEdit, ss.str().c_str());
    }

// Реализация функции PrintProcessList
void PrintProcessList(HWND hEdit) {
    HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnapshot == INVALID_HANDLE_VALUE) {
        MessageBox(nullptr, _T("Failed to take a snapshot of the processes."),
            szTitle, MB_OK);
        return;
    }

    PROCESSENTRY32 pe32;
    pe32.dwSize = sizeof(PROCESSENTRY32);

    std::wstringstream ss;

    if (Process32First(hSnapshot, &pe32)) {
        do {
            ss << pe32.szExeFile << _T(" (PID: ") << pe32.th32ProcessID <<
                _T(")\r\n");
        } while (Process32Next(hSnapshot, &pe32));
    }

    SetWindowText(hEdit, ss.str().c_str());

    CloseHandle(hSnapshot);
}

// Реализация функции PrintThreadList
void PrintThreadList(DWORD processID, HWND hEdit) {
    HANDLE hThreadSnap = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
    if (hThreadSnap == INVALID_HANDLE_VALUE) {
        MessageBox(nullptr, _T("Failed to take a snapshot of the threads."),
            szTitle, MB_OK);
        return;
    }

    THREADENTRY32 te32;
    te32.dwSize = sizeof(THREADENTRY32);

    std::wstringstream ss;

    if (Thread32First(hThreadSnap, &te32)) {
        do {
            if (te32.th32OwnerProcessID == processID) {
                ss << _T("Thread ID: ") << te32.th32ThreadID << _T("\r\n");
            }
        } while (Thread32Next(hThreadSnap, &te32));
    }

    SetWindowText(hEdit, ss.str().c_str());

    CloseHandle(hThreadSnap);
}

// Реализация функции PrintModuleList
void PrintModuleList(DWORD processID, HWND hEdit) {
    HANDLE hModuleSnap = CreateToolhelp32Snapshot(TH32CS_SNAPMODULE, processID);
    if (hModuleSnap == INVALID_HANDLE_VALUE) {
        MessageBox(nullptr, _T("Failed to take a snapshot of the modules."),
            szTitle, MB_OK);
        return;
    }
}

```

```

MODULEENTRY32 me32;
me32.dwSize = sizeof(MODULEENTRY32);

std::wstringstream ss;

if (Module32First(hModuleSnap, &me32)) {
    do {
        ss << _T("Module Name: ") << me32.szModule << _T("\r\n");
        ss << _T("Executable Path: ") << me32.szExePath << _T("\r\n\r\n");
    } while (Module32Next(hModuleSnap, &me32));
}

SetWindowText(hEdit, ss.str().c_str());

CloseHandle(hModuleSnap);
}

void FindProcess(DWORD processID, HWND hEdit) {
    HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnapshot == INVALID_HANDLE_VALUE) {
        MessageBox(nullptr, _T("Failed to take a snapshot of the processes."),
szTitle, MB_OK);
        return;
    }

    PROCESSENTRY32 pe32;
    pe32.dwSize = sizeof(PROCESSENTRY32);

    std::wstringstream ss;

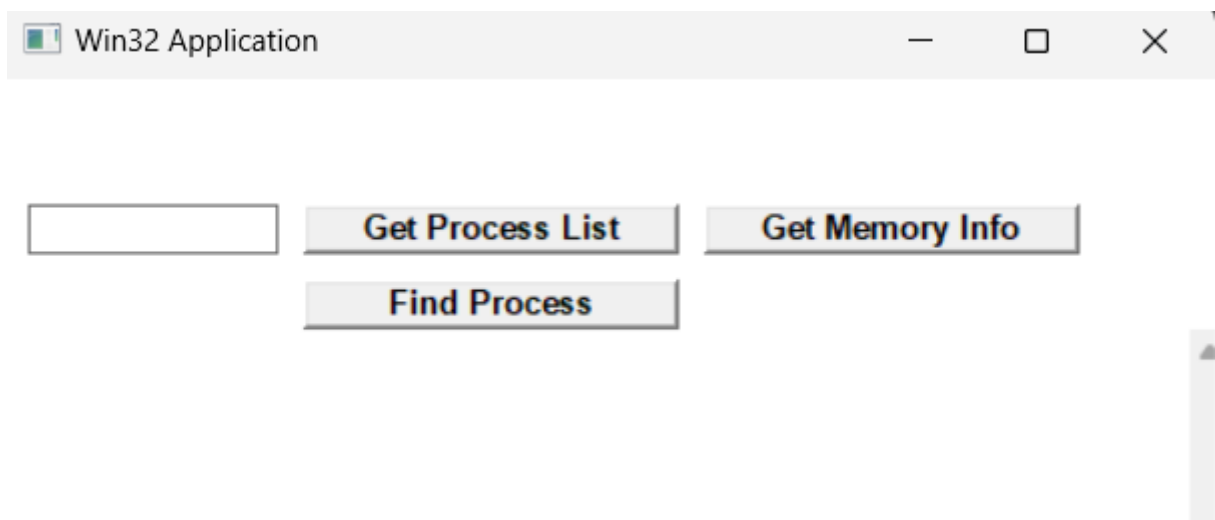
    if (Process32First(hSnapshot, &pe32)) {
        do {
            if (pe32.th32ProcessID == processID) {
                ss << pe32.szExeFile << _T(" (PID: ") << pe32.th32ProcessID <<
_T(")\r\n");
                break;
            }
        } while (Process32Next(hSnapshot, &pe32));
    }

    SetWindowText(hEdit, ss.str().c_str());

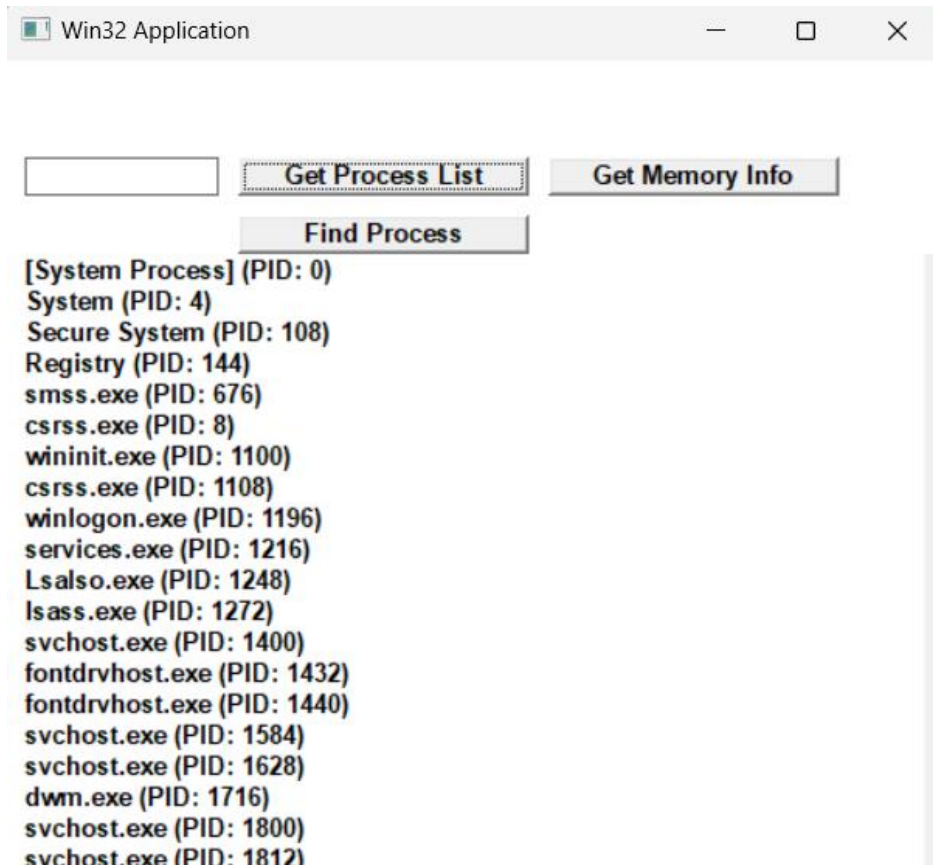
    CloseHandle(hSnapshot);
}

```

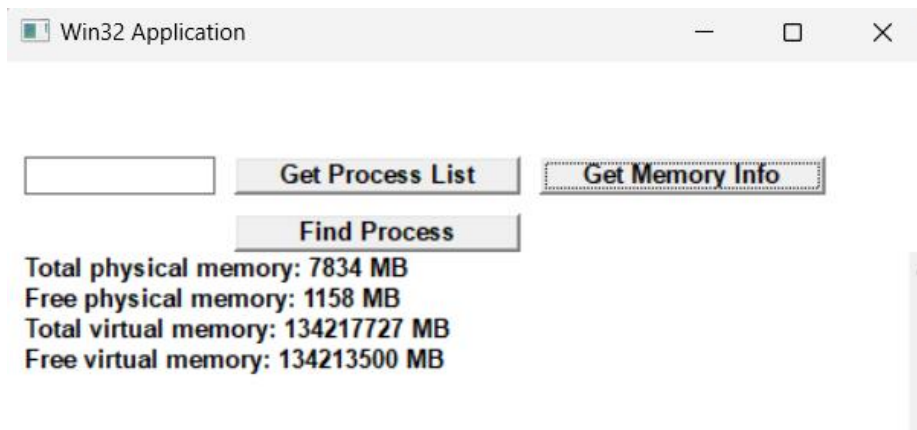
Вывод



Выводится окно, при нажатии кнопки Get Process List, выводится список всех процессов



При нажатии кнопки Get Memory Info выводится информация о памяти



Также можно найти конкретный процесс, написав его ID и нажав на кнопку Find Process

